МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра системного проектування

ЗВІТ
З ЛАБОРАТОРНОЇ РОБОТИ №2

з дисципліни «Професійний Пайтон»

на тему: «Структури даних та функціональна декомпозиція»

Виконав:
студент групи ФеП-32
Накладюк Володимир Тарасович
Варіант №15

Львів - 2026

1. Мета роботи

Сформувати практичні навички професійного використання базових і розширених структур даних Python (list, tuple, set, dict, модуль collections), навчитися обирати структуру відповідно до задачі, оцінювати складність основних операцій і декомпозувати програму на невеликі функції з чіткою відповідальністю, застосовуючи `*args`, `**kwargs`, lambda, closures та decorators.

2. Індивідуальне завдання

Варіант 15. Аналіз результатів наукових експериментів. Дані: назва експерименту, researcher, параметр і результат, дата. Необхідно реалізувати:

- множину унікальних researchers;
- пошук за researcher;
- середнє значення результатів, мінімальне та максимальне значення;
- фільтрацію результатів за заданим діапазоном;
- групування за researcher;
- формування рейтингу результатів;
- dict-index за ID або назвою експерименту.

3. Постановка задачі

Проєкт experiment-registry з лабораторної №1 розширено модулем обробки структурованих даних. Програма повинна містити щонайменше по одному list, tuple, set, dict і структуру з collections; list, set і dict comprehensions; не менше шести функцій, серед яких функції з `*args` і `**kwargs`; lambda, closure і decorator; реалізувати сортування, фільтрацію, групування, агрегацію та пошук, оцінити складність щонайменше трьох операцій і виконати benchmark пошуку в list і dict на 1 000, 10 000 та 100 000 записів.

4. Хід виконання

4.1. Структура проєкту

Порівняно з лабораторною №1 додано модулі `data.py`, `processors.py`, `analytics.py`, `decorators.py`, пакет `commands/` з підкомандами консольного застосунку та каталог `benchmarks/`. Точка входу `main.py` тепер розбирає аргументи через argparse і викликає потрібну підкоманду: `demo`, `menu` або `analyze`.

Лістинг 2.1. Структура проєкту lab02

```
lab02/
├── benchmarks/
│   └── benchmark_search.py
├── src/
│   └── experiment_registry/
│       ├── commands/
│       │   ├── __init__.py
│       │   ├── analyze.py
│       │   └── demo.py
```

```
│       ├── __init__.py
│       ├── analytics.py
│       ├── config.py
│       ├── data.py
│       ├── decorators.py
│       ├── exceptions.py
│       ├── main.py
│       ├── models.py
│       ├── processors.py
│       └── services.py
├── .gitignore
├── pyproject.toml
└── README.md
```

Таблиця 2.1. Нові модулі лабораторної №2

Модуль	Відповідальність
`data.py`	Вхідний набір даних як кортеж кортежів, перетворення в `Experiment`, генератор великих наборів для benchmark.
`processors.py`	Унікальні дослідники, dict-індекси, лінійний пошук, універсальні фільтр і сортування, групування, Counter, pipeline.
`analytics.py`	Середнє, мінімум/максимум, рейтинг, статистика за дослідниками, closures, функції з *args/**kwargs, експорт summary.
`decorators.py`	`measure_time`, параметризований `record_operation`, `require_non_empty`, історія операцій у deque.
`commands/analyze.py`	Сценарій лабораторної: послідовний виклик усіх операцій.
`benchmarks/ benchmark_search.py`	Вимірювання часу пошуку в list, dict, set.

4.2. Опис структур даних і пояснення вибору

Таблиця 2.2. Використані структури даних

Структура	Де використано	Чому обрано
`list`	Основний набір `list[Experiment]`, результати фільтрації, рейтинг	Упорядкована змінювана колекція; потрібні додавання, ітерація, сортування.
`tuple`	`RAW_EXPERIMENTS`, рядок даних, пара `(min, max)`, `(place, experiment)`, `ResearcherStats`	Незмінний запис фіксованої структури; захищає вхідні дані від випадкових змін, може бути ключем dict.
`set`	`get_unique_researchers`, `ids_set` у benchmark	Автоматично прибирає дублікати, перевірка належності O(1) у середньому.
`dict`	Індекси за ID і назвою, групування, статистика за дослідниками	Доступ за ключем O(1); природне відображення «ключ → значення».
`defaultdict`	`group_by_researcher`, `group_nested`	Не потрібно перевіряти наявність ключа перед додаванням.
`Counter`	`count_by_parameter`	Підрахунок частот і `most_common()` одним викликом.
`deque(maxlen=5)`	`OPERATION_HISTORY`	Обмежена історія останніх операцій, додавання/видалення з кінців O(1).

Лістинг 2.2. Вхідні дані як tuple (фрагмент data.py)

```
from experiment_registry.models import Experiment

# Raw rows are immutable tuples: (id, name, researcher, parameter, result, date).
RAW_EXPERIMENTS: tuple[tuple[int, str, str, str, float, str], ...] = (
    (1, "Thermal conductivity of Al alloy", "Olena Kovalenko", "temperature", 293.4,
"2026-02-03"),
    (2, "Gas compression test", "Taras Hnatiuk", "pressure", 101.3, "2026-02-05"),
```

```
    (3, "Thermal conductivity of Cu", "Olena Kovalenko", "temperature", 301.8, "2026-02-
10"),
```

4.3. Модуль collections

Групування реалізовано через `defaultdict(list)`: при першому зверненні до нового ключа автоматично створюється порожній список. Для вкладеного групування (researcher → parameter → результати) використано `defaultdict` з фабрикою-лямбдою. `Counter` рахує кількість експериментів за параметрами.

Лістинг 2.3. Використання defaultdict і Counter

```
def group_by_researcher(experiments: Iterable[Experiment]) -> dict[str, list[Experiment]]:
    """Group experiments by researcher using defaultdict(list). O(n)."""
    groups: defaultdict[str, list[Experiment]] = defaultdict(list)
    for item in experiments:
        groups[item.researcher].append(item)
    return dict(groups)


def group_nested(experiments: Iterable[Experiment]) -> dict[str, dict[str, list[float]]]:
    """Nested grouping: researcher -> parameter -> list of results."""
    groups: defaultdict[str, defaultdict[str, list[float]]] = defaultdict(lambda:
defaultdict(list))
    for item in experiments:
        groups[item.researcher][item.parameter].append(item.result)
    return {name: dict(params) for name, params in groups.items()}


def count_by_parameter(experiments: Iterable[Experiment]) -> Counter[str]:
    """Counter: number of experiments for each measured parameter."""
    return Counter(item.parameter for item in experiments)
```

4.4. Comprehensions

List comprehension використано для фільтрації, set comprehension — для унікальних дослідників, dict comprehension — для побудови індексів і статистики. Generator expression передається у `sum()`/`min()` без створення проміжного списку.

Лістинг 2.4. Set, dict і list comprehensions

```
def get_unique_researchers(experiments: Iterable[Experiment]) -> set[str]:
    """Set comprehension: unique researchers. O(n)."""
    return {item.researcher for item in experiments}


def build_index_by_id(experiments: Iterable[Experiment]) -> dict[int, Experiment]:
    """Dict comprehension: experiment_id -> experiment. Build O(n), lookup O(1)."""
    return {item.experiment_id: item for item in experiments}


def build_index_by_name(experiments: Iterable[Experiment]) -> dict[str, Experiment]:
    """Dict comprehension: lower-case name -> experiment."""
    return {item.name.lower(): item for item in experiments}


def find_by_researcher(experiments: Iterable[Experiment], researcher: str) ->
list[Experiment]:
    """List comprehension: all experiments of a researcher."""
    wanted = researcher.lower()
    return [item for item in experiments if item.researcher.lower() == wanted]
```

4.5. Основні функції, *args, **kwargs і lambda

Програму розбито на невеликі функції з однією відповідальністю (у модулях `processors.py` і `analytics.py` понад 20 функцій). Універсальний фільтр приймає довільну кількість предикатів через `*predicates`, універсальне сортування — довільну кількість полів через `*fields`; ключ сортування задано lambda-функцією.

*Лістинг 2.5. Універсальні фільтр, сортування та pipeline (*args, lambda)*

```
def filter_records(experiments: Iterable[Experiment], *predicates: Predicate) ->
list[Experiment]:
    """Universal filter: keep records that satisfy all predicates (*args)."""
    return [item for item in experiments if all(check(item) for check in predicates)]


def sort_records(experiments: Iterable[Experiment], *fields: str, reverse: bool = False)
-> list[Experiment]:
    """Universal sort by one or several attribute names. O(n log n)."""
    if not fields:
        fields = ("experiment_id",)
    return sorted(experiments, key=lambda item: tuple(getattr(item, f) for f in fields),
reverse=reverse)


def pipeline(data: Any, *steps: Callable[[Any], Any]) -> Any:
    """Apply processing steps one after another (functional pipeline)."""
    for step in steps:
        data = step(data)
    return data
```

*Лістинг 2.6. Функції з *args і **kwargs*

```
def average_of(*values: float) -> float:
    """Average of any number of positional values (*args)."""
    if not values:
        return 0.0
    return sum(values) / len(values)


def create_experiment(**fields: Any) -> Experiment:
    """Create an Experiment from keyword arguments (**kwargs)."""
    if isinstance(fields.get("date"), str):
        fields["date"] = date.fromisoformat(fields["date"])
    return Experiment(**fields)


def export_summary(experiments: list[Experiment], **options: Any) -> str:
    """Build a JSON summary; options: indent, top (size of ranking)."""
    indent = options.get("indent", 2)
    top = options.get("top", 3)
    bounds = min_max_result(experiments)
    summary = {
        "count": len(experiments),
        "researchers": sorted({item.researcher for item in experiments}),
        "average": round(sum(item.result for item in experiments) / len(experiments), 3)
        if experiments else None,
        "min_max": list(bounds) if bounds else None,
        "top": [
            {"place": place, "id": item.experiment_id, "result": item.result}
            for place, item in rank_results(experiments, top=top)
        ],
    }
    return json.dumps(summary, indent=indent, ensure_ascii=False)
```

4.6. Closures

Closure — це внутрішня функція, яка запам’ятовує змінні з області видимості зовнішньої функції після її завершення. `make_range_filter(low, high)` повертає предикат, що пам’ятає межі діапазону; `make_counter()` демонструє closure зі змінюваним станом через `nonlocal`. Значення, захоплені closure, зберігаються в атрибуті `__closure__` функції.

Лістинг 2.7. Closures у analytics.py

```
def make_range_filter(low: float, high: float) -> Callable[[Experiment], bool]:
    """Closure: predicate that remembers `low` and `high`."""
    if low > high:
        raise ValueError(f"Invalid range: {low} > {high}")

    def in_range(item: Experiment) -> bool:
        return low <= item.result <= high

    return in_range


def make_parameter_filter(parameter: str) -> Callable[[Experiment], bool]:
    """Closure: predicate for a measured parameter."""
    wanted = parameter.lower()
    return lambda item: item.parameter.lower() == wanted


def make_counter() -> Callable[[], int]:
    """Closure with mutable state via nonlocal: counts calls."""
    calls = 0

    def increment() -> int:
        nonlocal calls
        calls += 1
        return calls

    return increment
```

4.7. Decorators

Decorator — функція, яка приймає функцію і повертає нову з додатковою поведінкою, не змінюючи її коду. `measure_time` вимірює час виконання, `require_non_empty` повертає `None` для порожніх даних, а параметризований `record_operation(history, label)` — фабрика декораторів, що записує назву операції в `deque`. `functools.wraps` зберігає `__name__` і docstring оригінальної функції.

Лістинг 2.8. Decorators

```
def measure_time(func: Callable[P, R]) -> Callable[P, R]:
    """Print the execution time of the decorated function."""

    @wraps(func)
    def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
        start = perf_counter()
        result = func(*args, **kwargs)
        elapsed = perf_counter() - start
        print(f"[time] {func.__name__}: {elapsed * 1e6:.1f} us")
        return result

    return wrapper


def record_operation(history: deque[str], label: str | None = None) ->
```

```
Callable[[Callable[P, R]], Callable[P, R]]:
    """Parameterized decorator: append the operation name to `history` after each call."""

    def decorator(func: Callable[P, R]) -> Callable[P, R]:
        name = label or func.__name__

        @wraps(func)
        def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
            result = func(*args, **kwargs)
            history.append(name)
            return result

        return wrapper

    return decorator
```

Лістинг 2.9. Застосування кількох декораторів і формування рейтингу

```
@record_operation(OPERATION_HISTORY, "average")
@measure_time
@require_non_empty
def average_result(experiments: list[Experiment]) -> float:
    """Mean result of experiments (None for an empty list)."""
    return sum(item.result for item in experiments) / len(experiments)


@record_operation(OPERATION_HISTORY, "ranking")
def rank_results(experiments: Iterable[Experiment], top: int | None = None,
                 descending: bool = True) -> list[tuple[int, Experiment]]:
    """Ranking of results: list of (place, experiment) tuples."""
    ordered = sorted(experiments, key=lambda item: item.result, reverse=descending)
    ranking = list(enumerate(ordered, start=1))
    return ranking[:top] if top else ranking
```

4.8. Результат роботи програми

```
q3efirok@fedora: ~/Documents/Унік/ПП

(.venv) q3efirok@fedora:~/Documents/Унік/ПП/lab02$ experiment-registry analyze
Loaded experiments: 15 (list)
Unique researchers (5, set): ['Andrii Bondar', 'Iryna Melnyk', 'Maria Shevchuk', 'Olena Kovalenko',
'Taras Hnatiuk']
Index by ID, key 7 -> Heat exchanger efficiency
Index by name, 'gas compression test' -> #2

Experiments of Olena Kovalenko:
  #1   Thermal conductivity of Al alloy   Olena Kovalenko  temperature    293.40  2026-02-03
  #3   Thermal conductivity of Cu         Olena Kovalenko  temperature    301.80  2026-02-10
  #7   Heat exchanger efficiency          Olena Kovalenko  temperature    348.90  2026-03-02
  #12  Thermal shock resistance           Olena Kovalenko  temperature    412.50  2026-03-17
[time] average_result: 3.4 us

Temperature: average=260.56, (min, max)=(77.2, 412.5)
average_of(*args) for 3 values: 314.70

Temperature results in [250, 350] (closure filter):
  #1   Thermal conductivity of Al alloy   Olena Kovalenko  temperature    293.40  2026-02-03
  #3   Thermal conductivity of Cu         Olena Kovalenko  temperature    301.80  2026-02-10
  #7   Heat exchanger efficiency          Olena Kovalenko  temperature    348.90  2026-03-02
  #14  Thermocouple drift check           Maria Shevchuk   temperature    298.00  2026-03-24

Groups by researcher (defaultdict):
  Andrii Bondar    -> IDs [5, 10]
  Iryna Melnyk     -> IDs [4, 8, 11]
  Maria Shevchuk   -> IDs [13, 14, 15]
  Olena Kovalenko  -> IDs [1, 3, 7, 12]
  Taras Hnatiuk    -> IDs [2, 6, 9]

Nested grouping researcher -> parameter -> results:
  Andrii Bondar    {'temperature': [77.2, 92.1]}
  Iryna Melnyk     {'voltage': [3.71, 0.62, 0.71]}
  Maria Shevchuk   {'pressure': [26.5, 98.7], 'temperature': [298.0]}
  Olena Kovalenko  {'temperature': [293.4, 301.8, 348.9, 412.5]}
  Taras Hnatiuk    {'pressure': [101.3, 0.8, 250.4]}

Statistics by researcher (name, count, avg, min, max):
  Andrii Bondar    n=2  avg=   84.65  min=   77.20  max=   92.10
  Iryna Melnyk     n=3  avg=    1.68  min=    0.62  max=    3.71
  Maria Shevchuk   n=3  avg=  141.07  min=   26.50  max=  298.00
  Olena Kovalenko  n=4  avg=  339.15  min=  293.40  max=  412.50
  Taras Hnatiuk    n=3  avg=  117.50  min=    0.80  max=  250.40

Counter by parameter: [('temperature', 7), ('pressure', 5), ('voltage', 3)]

Ranking of temperature results (top 5):
  1.  412.50  Thermal shock resistance (Olena Kovalenko)
  2.  348.90  Heat exchanger efficiency (Olena Kovalenko)
  3.  301.80  Thermal conductivity of Cu (Olena Kovalenko)
  4.  298.00  Thermocouple drift check (Maria Shevchuk)
  5.  293.40  Thermal conductivity of Al alloy (Olena Kovalenko)

Pipeline: pressure -> sort by date desc -> first 3:
  #15  Pressure sensor validation         Maria Shevchuk   pressure        98.70  2026-03-27
  #13  Weather balloon ascent             Maria Shevchuk   pressure        26.50  2026-03-20
  #9   Hydraulic press calibration        Taras Hnatiuk    pressure       250.40  2026-03-06

Closure counter (nonlocal state) after 3 calls: 3
Operation history (deque, maxlen=5): ['average', 'min/max', 'researcher stats', 'ranking']

Export summary (**kwargs: indent=None, top=2):
{"count": 15, "researchers": ["Andrii Bondar", "Iryna Melnyk", "Maria Shevchuk", "Olena Kovalenko",
"Taras Hnatiuk"], "average": 153.776, "min_max": [0.62, 412.5], "top": [{"place": 1, "id": 12, "result":
 412.5}, {"place": 2, "id": 7, "result": 348.9}]}
```

Рис. 2.1. Результат роботи підкоманди analyze

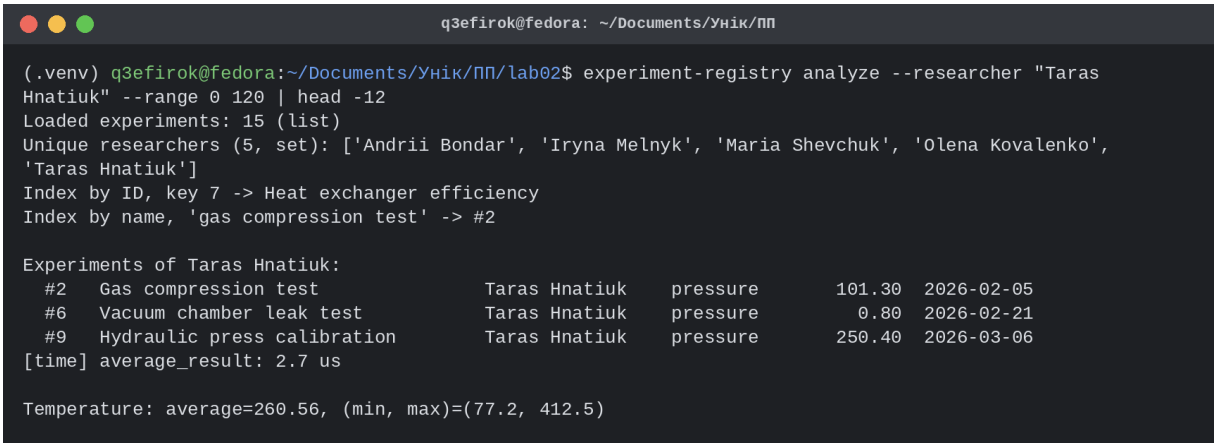


Рис. 2.2. Запуск з іншими параметрами (дослідник і діапазон)

4.9. Аналіз складності операцій

Таблиця 2.3. Таблиця складності операцій

Операція	Реалізація	Складність
Пошук експерименту за ID у list	`find_by_id_linear`	O(n)
Побудова dict-індексу	`build_index_by_id`	O(n)
Пошук за ключем у dict	`index.get(experiment_id)`	O(1) у середньому
Перевірка належності до set	`name in researchers`	O(1) у середньому
Унікальні дослідники	set comprehension	O(n)
Фільтрація / пошук за дослідником	list comprehension	O(n)
Групування	`defaultdict(list)`	O(n)
Мінімум / максимум / середнє	`min`, `max`, `sum`	O(n)
Рейтинг (сортування)	`sorted(..., key=lambda)`	O(n log n)

Лінійний пошук переглядає елементи по черзі, тому в найгіршому випадку виконує n порівнянь. Dict обчислює hash ключа і переходить одразу до потрібної комірки хеш-таблиці, тому середній час не залежить від кількості записів. Побудова індексу коштує O(n), але окуповується, якщо пошук виконується багато разів.

5. Експериментальна частина

Для benchmark згенеровано відтворювані набори даних (`generate_experiments`, seed=15) на 1 000, 10 000 і 100 000 записів. Для кожного розміру виконано 200 пошуків випадкових ID; час виміряно `timeit.repeat` (найкращий із 5 повторів) і перераховано на один пошук. Перед вимірюванням перевірено, що обидва способи повертають той самий об’єкт.

Лістинг 2.10. Фрагмент benchmark_search.py

```
def run() -> list[tuple[int, float, float, float, float, float]]:
    rows = []
    for size in SIZES:
        experiments = generate_experiments(size)
        rng = random.Random(size)
        targets = [rng.randint(1, size) for _ in range(LOOKUPS)]

        build_ms = min(timeit.repeat(lambda: build_index_by_id(experiments),
                                     number=1, repeat=REPEATS)) * 1e3
```

```
        index = build_index_by_id(experiments)
        ids_list = [item.experiment_id for item in experiments]
        ids_set = set(ids_list)

        list_search = best_per_lookup(
            lambda: [find_by_id_linear(experiments, t) for t in targets], LOOKUPS)
        dict_search = best_per_lookup(lambda: [index.get(t) for t in targets], LOOKUPS)
        list_in = best_per_lookup(lambda: [t in ids_list for t in targets], LOOKUPS)
        set_in = best_per_lookup(lambda: [t in ids_set for t in targets], LOOKUPS)

        assert all(find_by_id_linear(experiments, t) is index[t] for t in targets[:20])
        rows.append((size, list_search, dict_search, build_ms, list_in, set_in))
    return rows
```

```
q3efirok@fedora: ~/Documents/Унік/ПП

(.venv) q3efirok@fedora:~/Documents/Унік/ПП/lab02$ python benchmarks/benchmark_search.py
Lookups per run: 200, best of 5 runs, time per lookup in microseconds
  Records |  list search | dict search | dict build, ms |  x in list | x in set |  speedup
    1,000 |         5.18 |       0.038 |           0.02 |       1.90 |    0.035 |     135x
   10,000 |        55.50 |       0.042 |           0.39 |      19.96 |    0.038 |    1324x
  100,000 |       831.27 |       0.044 |           5.26 |     240.32 |    0.040 |   19097x
```

Рис. 2.3. Результати benchmark

Таблиця 2.4. Результати benchmark пошуку за ID

Кількість records	list search, мкс	dict search, мкс	побудова dict, мс	x in list, мкс	x in set, мкс	прискорення dict
1 000	5.18	0.038	0.02	1.90	0.035	135×
10 000	55.50	0.042	0.39	19.96	0.038	1324×
100 000	831.27	0.044	5.26	240.32	0.040	19097×

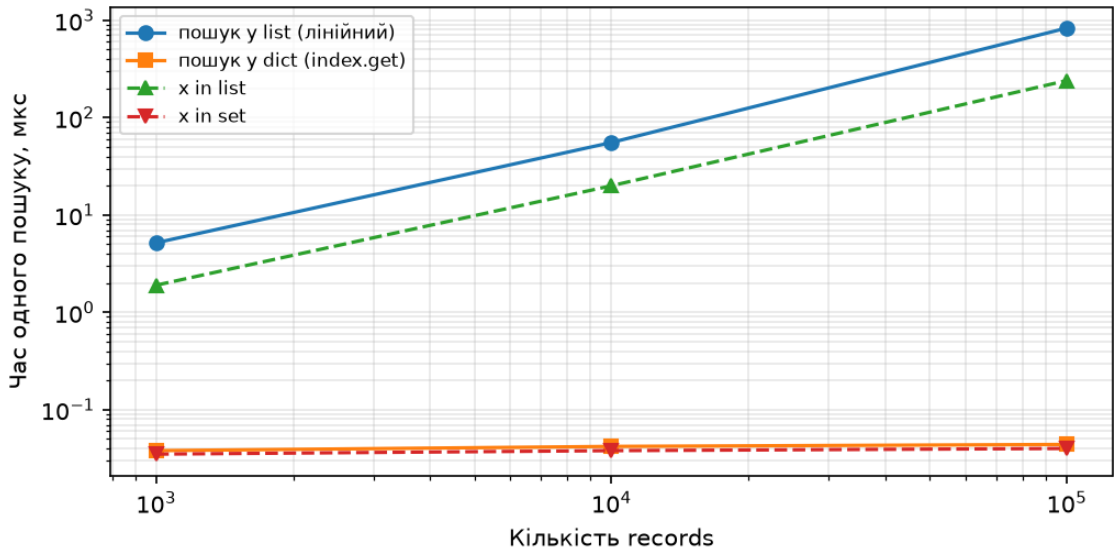


Рис. 2.4. Залежність часу пошуку від кількості записів (логарифмічні осі)

Час пошуку в list зростає приблизно пропорційно до кількості записів: при збільшенні розміру в 10 разів час також зростає приблизно в 10 разів, що відповідає O(n). Час пошуку в dict і перевірки належності до set практично сталий (десятки наносекунд) для всіх розмірів, що підтверджує середню складність O(1). На 100 000 записів dict швидший приблизно в 19097×. Побудова індексу коштує O(n) (5.26 мс

для 100 000 записів), тому вона вигідна, коли пошук виконується більше кількох разів.

5.1. Завдання підвищеної складності

- універсальна функція фільтрації `filter_records(*predicates)` і універсальне сортування `sort_records(*fields)`;
- параметризований decorator `record_operation(history, label)`;
- closures для створення фільтрів (`make_range_filter`, `make_parameter_filter`);
- вкладене групування `group_nested`;
- статистика через `Counter` і `deque` для історії останніх операцій;
- benchmark list vs set/dict;
- export summary у JSON (`export_summary`) і функція `pipeline` для послідовної обробки.

6. Висновок

У лабораторній роботі проєкт experiment-registry доповнено модулем обробки структурованих даних про експерименти. Кожну структуру обрано під задачу: list для впорядкованого набору експериментів, tuple для незмінних вхідних рядків і пар значень, set для унікальних дослідників, dict для індексів, групування і статистики, а `defaultdict`, `Counter` і `deque` спростили групування, підрахунок і ведення історії.

Програму декомпозовано на невеликі функції; продемонстровано comprehensions, `*args`, `**kwargs`, lambda, closures і decorators, зокрема параметризований. Benchmark показав, що пошук у list лінійно сповільнюється з ростом даних, тоді як пошук у dict залишається сталим, і на 100 000 записів dict швидший приблизно в 19097×. Отже, теоретична складність добре передбачає практичний час виконання, а правильний вибір структури даних дає виграш у тисячі разів без зміни алгоритму обробки.